



Chapter 24 - Quality Management

Review Techniques



Reviews and inspections

Reviews and inspections



- ✧ A group examines part or all of a process/system and its documentation
 - to find potential problems.
- ✧ Software/documents may be '**signed off**' at a review
 - progress to the next development stage has been approved by management.

Reviews and inspections



✧ There are different types of review with different objectives

- Inspections for **defect removal** (**product**);
- Reviews for **progress assessment** (**product and process**);
- **Quality reviews** (**product and standards**).

✧ **What can be reviewed?**

- Code, designs, specifications, test plans, standards, etc. can all be reviewed.

Phases in the review process



✧ Pre-review activities

- Pre-review activities are concerned with **review planning and review preparation**

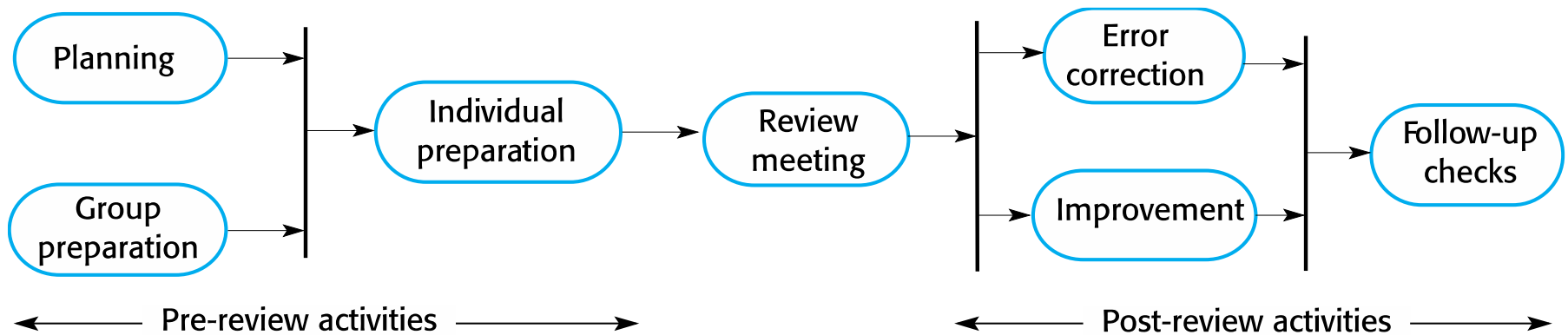
✧ The review meeting

- During the review meeting, an **author of the document or program being reviewed** 'walk through' the document with the review team.

✧ Post-review activities

- These **address the problems and issues** that have been raised during the review meeting.

The software review process



The software review process



✧ The processes suggested for reviews

- assume that the review team has [a face-to-face meeting](#) to discuss the software or documents that they are reviewing.

Distributed reviews



✧ Distributed reviews:

- project teams are now often distributed,
- sometimes **across countries or continents**
- it is impractical for team members to meet face to face.

✧ **Remote reviewing** can be supported using **shared documents** where each review team member can **annotate the document** with their comments.

Program inspections



- ✧ **Peer reviews** - engineers examine the source of a system with the aim of discovering anomalies and defects.
- ✧ **Inspections:**
 - not require execution of a system so may be used before implementation.
 - They may be applied to any representation of the system (requirements, design, configuration data, test data, etc.).
 - They have been shown to be an effective technique for discovering program errors.

An inspection checklist (a)



Fault class	Inspection check
Data faults	• Are all program variables initialized before their values are used? • Have all constants been named? • Should the upper bound of arrays be equal to the size of the array or Size -1? • If character strings are used, is a delimiter explicitly assigned? • Is there any possibility of buffer overflow?
Control faults	• For each conditional statement, is the condition correct? • Is each loop certain to terminate? • Are compound statements correctly bracketed? • In case statements, are all possible cases accounted for? • If a break is required after each case in case statements, has it been included?
Input/output faults	• Are all input variables used? • Are all output variables assigned a value before they are output? • Can unexpected inputs cause corruption?

An inspection checklist (b)



Fault class		Inspection check
Interface faults		• Do all function and method calls have the correct number of parameters? • Do formal and actual parameter types match? • Are the parameters in the right order? • If components access shared memory, do they have the same model of the shared memory structure?
Storage faults	management	• If a linked structure is modified, have all links been correctly reassigned? • If dynamic storage is used, has space been allocated correctly? • Is space explicitly deallocated after it is no longer required?
Exception faults	management	• Have all possible error conditions been taken into account?



Quality management and agile development

Quality management and agile development



✧ Quality management in agile development is

- **Informal** rather than document-based.
- It relies on establishing a **quality culture**, where all team members feel responsible for software quality.
- The agile community is fundamentally opposed to what it sees as the **bureaucratic overheads** of standards-based approaches and quality processes as embodied in ISO 9001.

Shared Good Practice



✧ *Check before check-in*

- Programmers are responsible for organizing their own code reviews with other team members before the code is checked in to the build system.

✧ *Never break the build*

- Team members should not check in code that causes the system to fail.
- Developers have to test their code changes against the whole system and be confident that these work as expected.

✧ *Fix problems when you see them*

- If a programmer discovers problems in code developed by someone else, they can fix these codes directly rather than referring them back to the original developer.

Reviews and agile methods



✧ In Scrum

- there is a **review meeting** after each iteration of the software has been completed (**a sprint review**), where quality issues and problems may be discussed.

✧ In Extreme Programming

- **pair programming** ensures that code is constantly being examined and reviewed by another team member.

Pair programming



- ✧ This is an approach where **2 people are responsible for code development** and work together to achieve this.
 - Code developed by an individual is therefore constantly being examined and reviewed by another team member.
 - Pair programming leads to **a deep knowledge of a program**, as both programmers have to understand the program in detail to continue development.

Pair programming weaknesses



✧ *Mutual misunderstandings*

- Both members of a pair may **make the same mistake** in understanding the system requirements.
 - Discussions may reinforce these errors.

✧ *Pair reputation*

- Pairs may be **reluctant to look for errors** because they do not want to slow down the progress of the project.

✧ *Working relationships*

- The pair's ability to discover defects is likely to be compromised by their **close working relationship** that often leads to **reluctance to criticize work partners**.

LTD: Agile QM and large systems



✧ Agile QM and large systems?



Software measurement

Software measurement



✧ Software measurement is concerned

- deriving a numeric value for an attribute of a software product or process.
- allows for objective comparisons between techniques and processes.

✧ Some organisations still don't make systematic use of software measurement.

Software metric



- ✧ **Any type of measurement which relates to a software system, process or related documentation**
- ✧ Lines of code in a program, the Fog index, number of person-days required to develop a component.
- ✧ Allow the software and the software process to be quantified.
- ✧ May be used to predict product attributes or to control the software process.
- ✧ **Product metrics** can be used for general predictions or to identify anomalous components.

LTD: Software metric



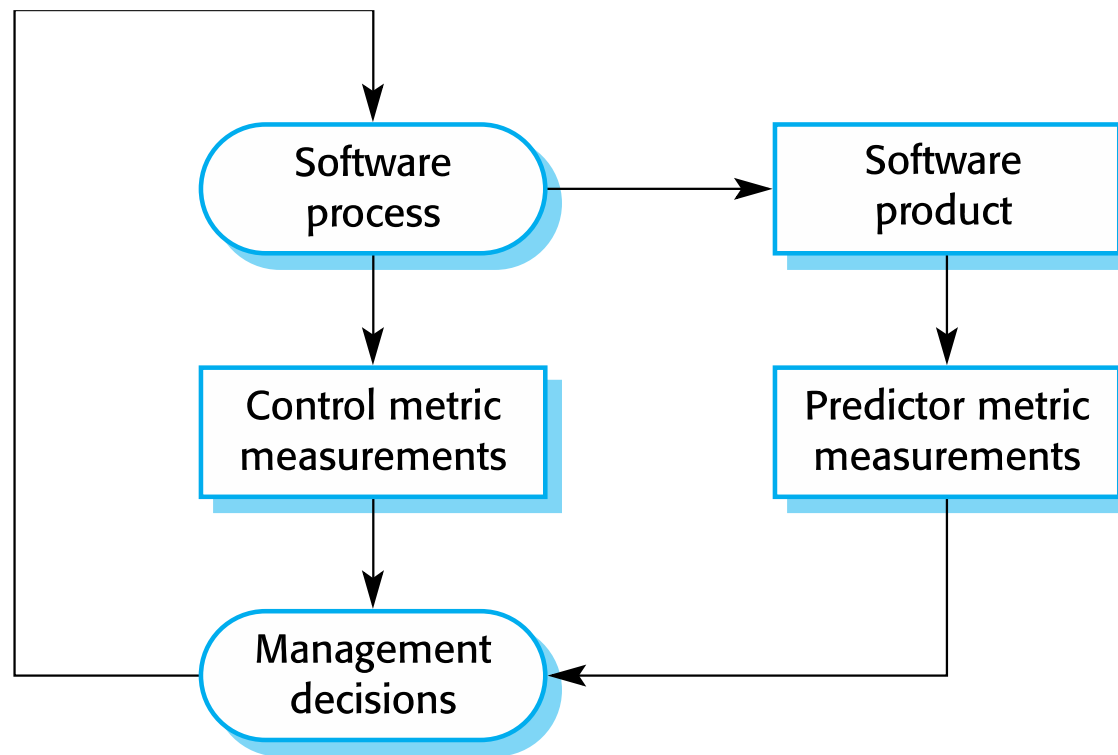
- ✧ Discuss in detail how Product metrics can be used for general predictions or to identify anomalous components.

Types of process metric



- ✧ *The time taken for a particular process to be completed*
 - This can be the **total time devoted to the process**, **calendar time**, the **time spent on the process** by particular engineers, and so on.
- ✧ *The resources required for a particular process*
 - Resources might include **total effort in person-days**, **travel costs** or **computer resources**.
- ✧ *The number of occurrences of a particular event*
 - Examples of events that might be monitored include the **number of defects discovered during code inspection**, the **number of requirements changes** requested, the **number of bug reports** in a delivered system and the **average number of lines of code** modified in response to a requirements change.

Predictor and control measurements



Use of measurements



✧ To assign a value to system quality attributes

- By measuring the characteristics of system components, such as their **cyclomatic complexity**, and then **aggregating these measurements**, you can assess system quality attributes, such as **maintainability**.

✧ To identify the system components whose quality is sub-standard

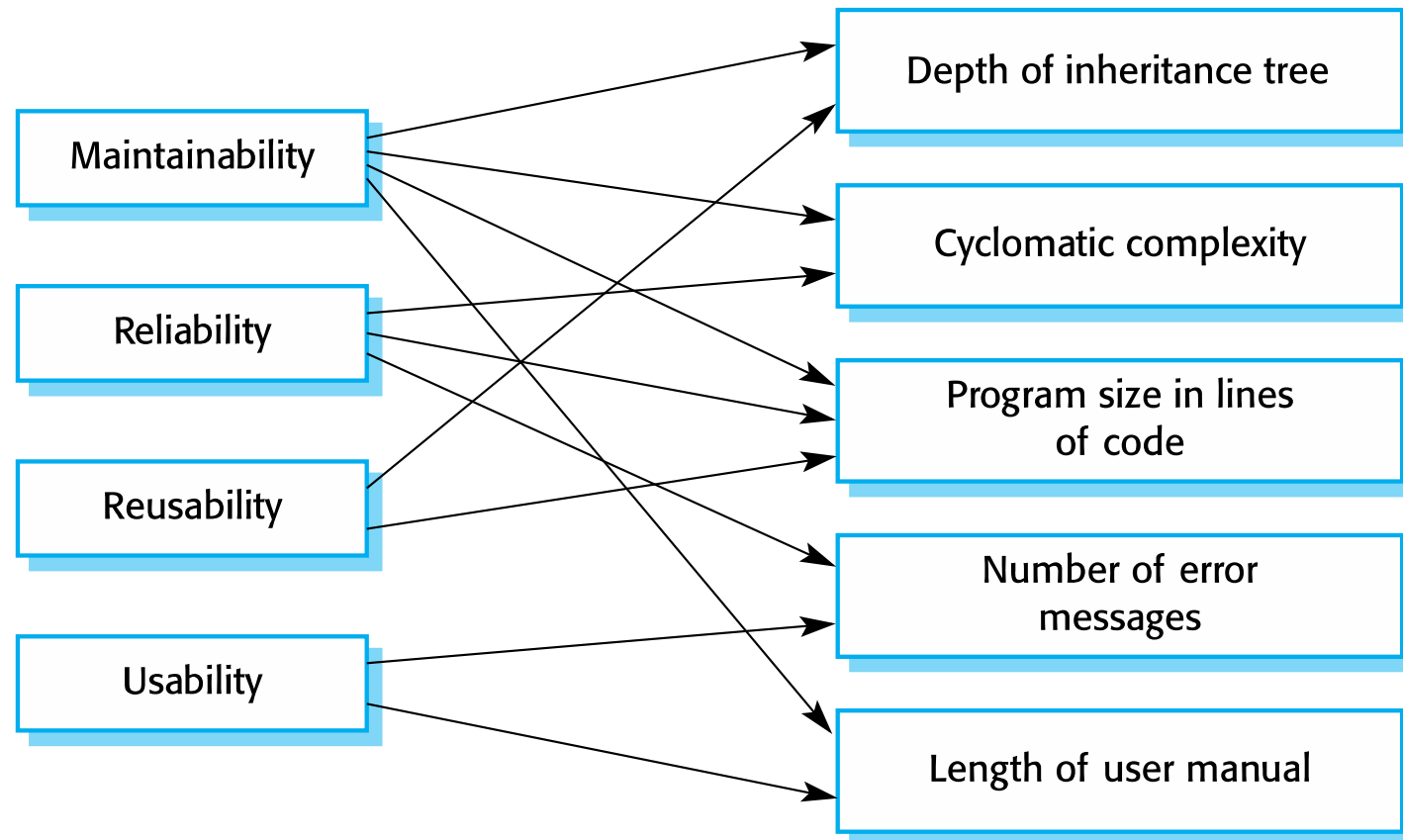
- Measurements can **identify individual components with characteristics that deviate from the norm**.

LTD: Relationships between internal and external software



External quality attributes

Internal attributes



LTD: Problems with measurement in industry



✧ ?

✧ Discuss the Problems with measurement in industry

Empirical software engineering



- ✧ Software measurement and metrics are the **basis of empirical software engineering**.
- ✧ This is a **research area** in which experiments on software systems and the **collection of data about real projects has been used to form and validate hypotheses about software engineering** methods and techniques.
- ✧ **Research on empirical software engineering has not had a significant impact on software engineering practice.**
 - It is difficult to relate generic research to a project that is different from the research study.

Product metrics



- ✧ A quality metric should be a **predictor of product quality**.
- ✧ Classes of product metric
 - **Dynamic metrics** which are collected by measurements made of a **program in execution**;
 - are **closely related** to software quality attributes
 - Dynamic metrics help assess **efficiency** and **reliability**
 - **Static metrics** which are collected by measurements made of the **system representations**;
 - an **indirect relationship** with quality attributes
 - Static metrics help assess **complexity**, **understandability** and **maintainability**.

Static software product metrics



Software metric	Description
Fan-in/Fan-out	Fan-in is a measure of the number of functions or methods that call another function or method (say X). Fan-out is the number of functions that are called by function X. A high value for fan-in means that X is tightly coupled to the rest of the design and changes to X will have extensive knock-on effects. A high value for fan-out suggests that the overall complexity of X may be high because of the complexity of the control logic needed to coordinate the called components.
Length of code	This is a measure of the size of a program . Generally, the larger the size of the code of a component, the more complex and error-prone that component is likely to be. Length of code has been shown to be one of the most reliable metrics for predicting error-proneness in components .

Static software product metrics



Software metric	Description
Cyclomatic complexity	This is a measure of the control complexity of a program. This control complexity may be related to program understandability.
Length of identifiers	This is a measure of the average length of identifiers (names for variables, classes, methods, etc.) in a program. The longer the identifiers, the more likely they are to be meaningful and hence the more understandable the program.
Depth of conditional nesting	This is a measure of the depth of nesting of if-statements in a program. Deeply nested if-statements are hard to understand and potentially error-prone.
Fog index	This is a measure of the average length of words and sentences in documents. The higher the value of a document's Fog index, the more difficult the document is to understand.

LTD: Fog index



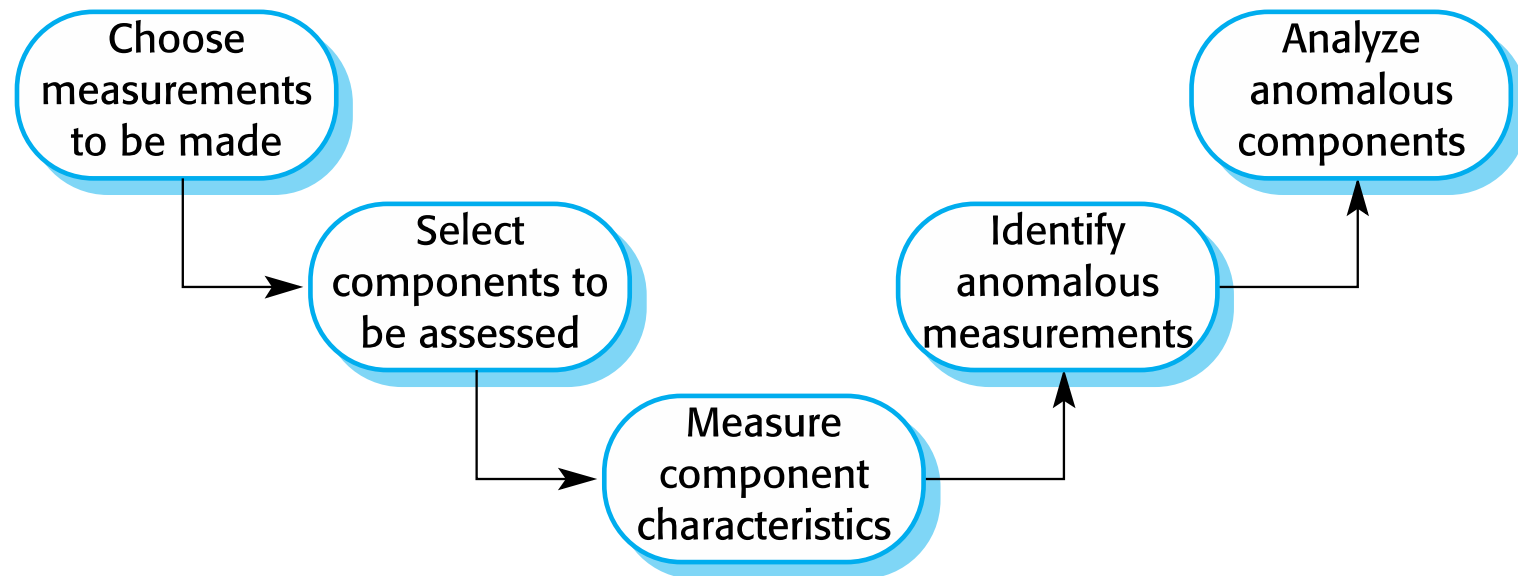
- ✧ With illustration, discuss and calculate a Fog index using numerical examples!
- ✧ Discuss the CK object-oriented metrics suite

Software component analysis



- ✧ System **component can be analyzed separately** using a range of metrics.
- ✧ The **values of these metrics may then compared for different components** and, perhaps, with
 - Also, perhaps, with historical measurement data collected on previous projects.

The process of product measurement



Measurement ambiguity



- ✧ When you **collect quantitative data** about software and software processes, you have to analyze that data to understand its meaning.
- ✧ It is easy to **misinterpret data** and to make inferences that are incorrect.
- ✧ You **cannot simply look at the data on its own**.
 - You must also consider the context where the data is collected.

LTD: Software analytics



✧ *Software analytics is analytics on software data for managers and software engineers with the aim of empowering software development individuals and teams to gain and share insight from their data to make better decisions.*